ESP32-Based Smart Lock Using FreeRTOS

Project Technical Report — Application Firmware and RTOS Demonstration Experiments

Project	ESP32 FreeRTOS Smart Lock
Platform	ESP32-WROOM-32 / ESP32 DevKit
Deliverables	Smart-lock application sketch + separate RTOS demonstration sketch

Prepared as a technical explanation for academic project evaluation

Abstract

This project is an ESP32-based smart-lock prototype using FreeRTOS. It combines RFID card access, keypad PIN access, a concealed administrator override button, a servo-operated lock, and an OLED status display. FreeRTOS separates the system into independently scheduled tasks. Queues transfer access requests and lock commands, a mutex protects shared state, task notifications wake tasks, and a one-shot software timer triggers automatic relocking.

A second, independent sketch demonstrates race conditions, priority inversion, and deadlock with timeout-based recovery. These deliberately controlled experiments are kept separate from the normal lock firmware.

1. Problem and Motivation

A basic smart lock can be written using a single Arduino loop. However, input scanning, authentication, display refresh, diagnostics, and timed relocking then share one execution flow. A long operation can delay another operation unless the program is carefully designed as a non-blocking state machine.

This project uses FreeRTOS to give independent activities their own tasks and to demonstrate practical concurrency, inter-task communication, synchronization, task priorities, and timed behavior. The RTOS is used to structure the application, not merely to add complexity.

2. Objectives

- Provide RFID and keypad-based access requests.
- Provide a concealed long-press administrator override for demonstration.
- Centralize access decisions in a security task.
- Make one task the exclusive owner of servo actuation.
- Relock automatically without blocking other application tasks.
- Refresh the OLED and report system diagnostics periodically.
- Demonstrate queues, mutexes, task notifications, software timers, task priorities, and periodic scheduling.
- Demonstrate race conditions, priority inversion, and deadlock separately.

Device / signal	GPIO	Purpose / notes
RC522 SCK	18	SPI clock
RC522 MISO	19	SPI data to ESP32
RC522 MOSI	23	SPI data from ESP32
RC522 SDA / SS	5	SPI chip select
RC522 RST	27	Reader reset
OLED SDA / SCL	21 / 22	I²C; configured address 0x3C
Servo signal	13	PWM control
RFID request button	34	External pull-up; button to GND
Keypad request button	35	External pull-up; button to GND
Admin override button	39	External pull-up; button to GND
Keypad rows	4, 14, 16, 17	Matrix rows
Keypad columns	25, 26, 32, 33	Matrix columns

3. Hardware and Pin Assignment

The intended target is a classic ESP32-WROOM-32 / ESP32 DevKit. The hardware includes an MFRC522 RFID reader, 4×4 matrix keypad, I²C SSD1306 OLED, SG90-class servo, and three push buttons.

GPIO 34, 35, and 39 are input-only on the classic ESP32 and do not have internal pull-up resistors. Use external pull-up resistors (for example, 10 kΩ) and connect each button between its GPIO and GND. The RC522 uses 3.3 V logic. Power the servo from a suitable external regulated supply and connect its ground to ESP32 GND. Confirm the OLED address and board pin/strapping behavior during bring-up.

4. Functional Operation

RFID: The RFID request button wakes RFIDTask. The task opens a scan window, reads a card UID, compares it with the configured allowed UID, and submits an allow/deny request.

Keypad: The keypad request button starts a PIN-entry session. Numeric keys enter digits, '*' clears the entry, and '#' submits it. The session expires after its configured time limit.

Admin override: Holding the concealed button for the configured duration submits an administrator override request. In this prototype it bypasses the demonstration lockout; this is not production-grade administrator authentication.

Security decision: SecurityTask receives access requests, applies the credential result and failed-attempt lockout policy, updates shared state, and sends an unlock command when access is granted.

Lock actuation: LockTask is the only application task that commands the servo. It moves the servo to the unlock position and starts a one-shot software timer. When the timer expires, its callback notifies LockTask, which returns the servo to the locked position.

Display and diagnostics: DisplayTask periodically reads a protected snapshot of application state and refreshes the OLED. DiagnosticsTask periodically prints free heap, minimum-ever free heap, queue occupancy, and selected task stack high-water marks.

Task	Priority	Responsibility
InputTask	3	Button debounce; wakes RFID/keypad tasks; detects admin long press.
RFIDTask	2	Card scan and UID result submission.
KeypadTask	2	PIN-entry session and result submission.
SecurityTask	4	Access decision, failed-attempt count, lockout, and unlock command.
LockTask	3	Exclusive servo owner; unlock timing and automatic relock.
DisplayTask	1	Periodic OLED refresh using vTaskDelayUntil().
DiagnosticsTask	1	Periodic heap, queue, and stack diagnostics.

5. FreeRTOS Task Architecture

The application sketch has seven application tasks. Arduino loop() is intentionally idle; application behavior is handled by FreeRTOS tasks.

6. Inter-Task Communication and Synchronization

Queues: accessQueue carries structured access requests to SecurityTask. lockQueue carries lock commands from SecurityTask to LockTask. This separates authentication decisions from physical actuator control.

Mutex: stateMutex protects the shared AppState structure, including lock status, counters, status text, and the latest event. Tasks take the mutex before reading or modifying shared fields.

Task notifications: InputTask wakes RFIDTask and KeypadTask when their request buttons are pressed. The software timer callback notifies LockTask when the unlock interval expires. The callback does not operate the servo directly.

Software timer: unlockTimer is a one-shot timer. It allows automatic relocking without placing a long blocking delay inside LockTask.

Periodic scheduling: DisplayTask and DiagnosticsTask use vTaskDelayUntil() to execute at regular intervals.

Priorities: SecurityTask has the highest application priority because it processes access decisions. InputTask and LockTask are above the authentication and background display/diagnostics tasks. These are initial design choices, not measured real-time guarantees; response times and stack usage must be validated on hardware.

7. Separate RTOS Demonstration Sketch

The second sketch is independent of normal lock operation. Upload it separately and use Serial Monitor at 115200 baud. Commands are R (race condition), I (priority inversion), D (deadlock/recovery), and H (help).

Race condition: Two tasks each increment a shared counter 100,000 times. The unsafe version deliberately separates the read-modify-write sequence with a task yield, allowing interleaving and lost updates. The protected version surrounds the increment with a mutex. Expected result is 200,000. The unsafe result can vary; repeat the test because an individual run may occasionally reach the expected value.

Priority inversion: Low-, medium-, and high-priority tasks are pinned to the same ESP32 core for a more controlled experiment. Low owns a lock; High requests it and blocks; Medium performs CPU work. The first run uses a binary semaphore, which has no priority inheritance. The second uses a FreeRTOS mutex, which supports priority inheritance. With the mutex, Low can temporarily inherit High's priority and release the resource sooner. Timing is illustrative and depends on scheduler and board-package behavior.

Deadlock/recovery: Task A acquires mutex A and task B acquires mutex B. Each then requests the other's mutex, creating circular wait. The demonstration uses bounded timeouts; a task that cannot acquire the second mutex reports the timeout and releases its first mutex. This demonstrates a deadlock condition and timeout-based recovery, not a permanent system hang.

8. Why FreeRTOS Is Appropriate Here

The system has activities with different timing and responsiveness needs. RFID scanning and PIN entry should not prevent display refresh. Automatic relocking should occur without blocking authentication. Diagnostics should run periodically without taking over application control. Queues provide structured communication between tasks, while a mutex protects shared state.

A single-loop implementation is possible for a small lock. FreeRTOS is useful here because it makes concurrency, task ownership, timing, and synchronization explicit and demonstrable. This project does not claim that every smart lock requires an RTOS.

9. Limitations and Security Considerations

• RFID UID and PIN are compiled into the sketch. Static UIDs may be cloned; a hard-coded PIN is not secure credential storage.
• The admin override is a long-press button and bypasses the demonstration lockout. It is not strong administrator authentication.
• There is no physical lock-position or door-contact sensor. Software state reflects the commanded servo position, not independently verified mechanical engagement.
• The current OLED shows the latest event and counters; it does not implement persistent storage or the earlier ten-entry history viewer.
• The lockout is a demonstration policy. A production system needs carefully specified expiry, reset, power-loss, and audit behavior.
• The code is a teaching prototype and has not been compiled or validated on final hardware as part of this report. Library versions, board package, wiring, servo mechanics, and task stack sizes must be verified.
• Diagnostics are not a watchdog or a complete health supervisor. Watchdog configuration and Event Groups are intentionally omitted from this version.

10. Verification Plan

1. Compile each sketch independently with the selected ESP32 board package and required libraries.
2. Verify OLED initialization (configured address 0x3C) and RFID UID reading.
3. Verify all buttons and external pull-ups on GPIO 34, 35, and 39.
4. Replace placeholder UID DEADBEEF and test accepted and rejected cards.
5. Test correct/incorrect PIN, '*', '#', and PIN-session timeout.
6. Test failed-attempt lockout and administrator override separately.
7. Confirm only LockTask commands the servo and automatic relocking works.
8. Observe OLED and diagnostics during RFID/PIN activity; inspect stack high-water marks and free heap.
9. Run R, I, and D in the separate demo sketch. Repeat the race experiment and compare semaphore/mutex priority-inversion timings.
10. Record compile results, measured timings, wiring changes, and observed behavior before claiming hardware validation.

11. Conclusion

The project demonstrates a task-based smart-lock architecture on ESP32 using FreeRTOS. Authentication, access decisions, lock actuation, display updates, and diagnostics are separated into independently scheduled tasks. Queues, a mutex, task notifications, a software timer, priorities, and periodic scheduling each serve a defined purpose. A second sketch isolates race-condition, priority-inversion, and deadlock/recovery experiments so that concurrency behavior can be demonstrated without deliberately destabilizing the smart-lock firmware.

The implementation is an academic prototype. Hardware validation, stronger credential handling, mechanical feedback, and reliability testing remain necessary before any real security deployment.